

Statistics 4420: Lecture 23

Hidden Markov Models Part 2

(Not in Gelman Book)

Paul Sabin

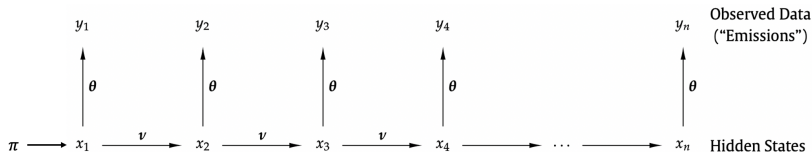
Department of Statistics, The Wharton School

psabin88@wharton.upenn.edu

April 11, 2024

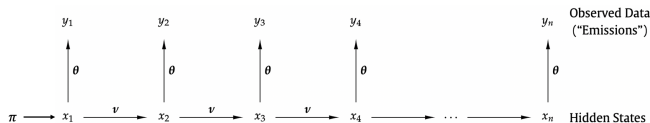
Credit to Zehang Chen (2020) & Shane Jensen for his help with course notes

Hidden Markov Models are an extension of mixture models where the **hidden state evolves over time**



In a hidden Markov model, the observed data are called **emissions** from the hidden state variables, and these hidden state variables have a **Markovian relationship**:

$$x_t \mid x_{t-1} \perp x_1, x_2, \dots, x_{t-2}$$



In addition to hidden states $\mathbf{x} = (x_1, \dots, x_n)$, we have several sets of **unknown parameters**:

θ are parameters for the **emission model** $p(y_t | x_t, \theta)$

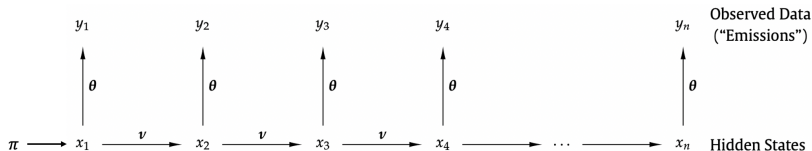
ν, π are parameters for the **state model** $p(x_t | x_{t-1}, \omega)$

State model parameters consist of **transition probabilities** ν

$$\nu(i, j) = p(x_t = j | x_{t-1} = i)$$

and **initial state probabilities** π

$$\pi_i = p(x_1 = i)$$



Work in R: Example of a Hidden Markov Models with a Continuous Outcome (Emission)

Continuous emission HMM: emissions y_t are **Normal** outcomes with different means from two hidden states x_t

Our goal today is to get samples from the **joint posterior** distribution of all unknown parameters (including hidden states):

$$\begin{aligned} p(\mathbf{x}, \boldsymbol{\theta}, \boldsymbol{\omega} \mid \mathbf{y}) &\propto p(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta}) \cdot p(\mathbf{x} \mid \boldsymbol{\omega}) \cdot p(\boldsymbol{\theta}, \boldsymbol{\omega}) \\ &\propto \prod_{t=1}^n \underbrace{p(y_t \mid x_t, \boldsymbol{\theta})}_{\text{emission model}} \cdot \underbrace{p(x_t \mid x_{t-1}, \boldsymbol{\omega})}_{\text{state model}} \cdot \underbrace{p(\boldsymbol{\theta}, \boldsymbol{\omega})}_{\text{prior}} \end{aligned}$$

In order to do **full posterior sampling**, we will build upon some historical estimation techniques for hidden Markov models, including the **Viterbi algorithm** we explored last class

Early methods for HMMs focussed on **point estimation**:

1. Finding optimal **“path” of hidden states** conditional on particular values of model parameters (θ, ω) :

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmax}} p(\mathbf{x}, \mathbf{y} \mid \theta, \omega)$$

2. Estimating optimal emission and state model parameters:

$$(\hat{\theta}, \hat{\omega}) = \underset{\theta, \omega}{\operatorname{argmax}} p(\mathbf{y} \mid \theta, \omega)$$

which involves integrating the hidden states $\mathbf{x}$ out of the likelihood $p(\mathbf{y} \mid \theta, \omega) = \sum_{\mathbf{x}} p(\mathbf{y}, \mathbf{x} \mid \theta, \omega)$

We will first review (1) from last class and then adapt it for (2) which will help with **full posterior sampling**

Conditional on particular values of the model parameters (θ, ω) , we want to find the path $\hat{\mathbf{x}}$ through the hidden states that maximizes the data likelihood:

$$\hat{\mathbf{x}} = \underset{\mathbf{x}}{\operatorname{argmax}} p(\mathbf{x}, \mathbf{y} \mid \theta, \omega)$$

The **Viterbi algorithm** we covered last class is a **dynamic programming** to **recursively calculate** $\max_{\mathbf{x}} p(\mathbf{x}, \mathbf{y} \mid \theta, \omega)$ and then backtrack to find the optimal path $\hat{\mathbf{x}}$

We first define a set of variables $\delta(t, i)$ that will store our **recursive calculations**:

$$\delta(t, i) = \max_{x_1, \dots, x_{t-1}} p(x_1, \dots, x_{t-1}, x_t = i, y_1, y_2, \dots, y_t \mid \theta, \omega)$$

where t indexes the time point and i indexes the different states that can be taken on by x_t

We start the Viterbi algorithm at $t = 1$ by calculating the likelihood of y_1 over each possible value of hidden state x_1 :

$$\begin{aligned} \text{Initialization :} \quad \delta(1, i) &= p(y_1 \mid x_1 = i, \theta) \cdot p(x_1 = i \mid \omega) \\ &= p(y_1 \mid x_1 = i, \theta) \cdot \pi_i \end{aligned}$$

We then **recursively maximize** over all possible values of each hidden state as we **move forward in time**

Forward Induction: for $t = 2, \dots, n$

$$\begin{aligned}\delta(t, i) &= \max_j p(\mathbf{y}_{1:t-1}, \mathbf{x}_{1:t-1}) \cdot p(x_t = i | x_{t-1} = j) \cdot p(y_t | x_t = i) \\ &= \max_j \delta(t-1, j) \cdot \nu(j, i) \cdot p(y_t | x_t = i)\end{aligned}$$

This recursive calculation ends at the final time point $t = n$ at which point we have maximized over all hidden states:

$$\max_{\mathbf{x}} p(\mathbf{x}, \mathbf{y} | \boldsymbol{\theta}, \boldsymbol{\omega}) = \max_i \delta(n, i)$$

Now that we have maximized $p(\mathbf{x}, \mathbf{y} \mid \boldsymbol{\theta}, \boldsymbol{\omega})$, we **trace back through our recursive calculations** to figure out the specific values of each hidden state that lead to that maximal value

Starting with the final time point,

$$\hat{x}_n = \operatorname{argmax}_i \delta(n, i)$$

we then trace backwards in time to find the **optimal hidden state** at each time point

Backtrack: for $t = n - 1, n - 2, \dots, 1$:

$$\hat{x}_t = \operatorname{argmax}_i \delta(t, i) \cdot \nu(i, \hat{x}_{t+1})$$

Let's return to our example:

Continuous emission HMM: emissions y_t are **Normal** outcomes with different means from two hidden states x_t

Work in R:
Viterbi Algorithm applied to
HMM Examples

The Viterbi algorithm is a computationally efficient method for calculating the optimal path $\hat{\mathbf{x}}$ through the hidden states using **dynamic programming**

However, there are two key **disadvantages** to the Viterbi approach:

1. The Viterbi algorithm depends on values for θ and ω but these **parameters are usually unknown** in practice
2. The optimal path $\hat{\mathbf{x}}$ calculated by Viterbi is only a point estimate and we want to **estimate our uncertainty** in $\mathbf{x}$ as well

The Bayesian approach corrects both of these issues by doing full posterior sampling which also involves **dynamic programming**

Back to the second historical method mentioned earlier, which is estimating optimal emission and state model parameters by **integrating over all hidden states**:

$$p(\mathbf{y} | \theta, \omega) = \sum_{\mathbf{x}} p(\mathbf{y}, \mathbf{x} | \theta, \omega)$$

Note that trying to do this sum **exhaustively** over all possible configurations of $\mathbf{x}$ is computationally impractical

- ▶ if we have n time points and hidden states x_t take on m values, then we have m^n possible configurations of $\mathbf{x}$

Instead, we can use **dynamic programming** by adapting the Viterbi algorithm to do **recursive summing** instead of maximizing

Similar to the **Viterbi algorithm**, first define a set of variables $\alpha(t, i)$ that will store our recursive calculations:

$$\alpha(t, i) = \sum_{x_1, \dots, x_{t-1}} p(x_1, \dots, x_{t-1}, x_t = i, y_1, y_2, \dots, y_t \mid \theta, \omega)$$

where t indexes the time point and i indexes the different states that can be taken on by x_t

We start this **forward summing algorithm** at $t = 1$ by calculating the likelihood of y_1 over each possible value of hidden state x_1 :

$$\begin{aligned} \textbf{Initialization :} \quad \alpha(1, i) &= p(y_1 \mid x_1 = i, \theta) \cdot p(x_1 = i \mid \omega) \\ &= p(y_1 \mid x_1 = i, \theta) \cdot \pi_i \end{aligned}$$

We then **recursively sum** over all possible values of each hidden state as we **move forward in time**

Forward Induction: for $t = 2, \dots, n$

$$\begin{aligned}\alpha(t, i) &= \sum_j p(\mathbf{y}_{1:t-1}, \mathbf{x}_{1:t-1}) \cdot p(x_t = i \mid x_{t-1} = j) \cdot p(y_t \mid x_t = i) \\ &= \sum_j \alpha(t-1, j) \cdot \nu(j, i) \cdot p(y_t \mid x_t = i)\end{aligned}$$

This recursive calculation ends at the final time point $t = n$ at which point we have summed over all hidden states:

$$p(\mathbf{y} \mid \boldsymbol{\theta}, \boldsymbol{\omega}) = \sum_i \alpha(n, i)$$

We need those forward summing recursive calculations as part of a **Gibbs Sampler** for getting samples from the **joint posterior**:

1. Sample a path through **hidden states**

$$\mathbf{x} \sim p(\mathbf{x} | \boldsymbol{\theta}, \boldsymbol{\nu}, \boldsymbol{\pi}, \mathbf{y})$$

2. Sample **emission model** parameters

$$\boldsymbol{\theta} \sim p(\boldsymbol{\theta} | \mathbf{x}, \boldsymbol{\nu}, \boldsymbol{\pi}, \mathbf{y})$$

3. Sample **state model** parameters

$$\boldsymbol{\nu} \sim p(\boldsymbol{\nu} | \mathbf{x}, \boldsymbol{\theta}, \mathbf{y})$$

$$\boldsymbol{\pi} \sim p(\boldsymbol{\pi} | \mathbf{x}, \boldsymbol{\theta}, \mathbf{y})$$

In order to sample a **full path through the hidden states**

$$\mathbf{x} \sim p(\mathbf{x} | \boldsymbol{\theta}, \nu, \boldsymbol{\pi}, \mathbf{y})$$

we use the **forward summing** algorithm to recursively calculate

$$\alpha(t, i) = \sum_{x_1, \dots, x_{t-1}} p(x_1, \dots, x_{t-1}, x_t = i, y_1, y_2, \dots, y_t | \boldsymbol{\theta}, \boldsymbol{\omega})$$

for all time points $t = 1, \dots, n$

We will then sample the hidden states x_t for each time point t
moving backwards in time from $t = n$ to $t = 1$

Once we have $\alpha(t, i)$ calculated for all time points $t = 1, \dots, n$ by the **forward summing algorithm**, we sample the hidden state at the final time point:

$$\text{Sample } x_n = i \text{ with probability } p(x_n = i) = \frac{\alpha(n, i)}{\sum_j \alpha(n, j)}$$

and then start **moving backwards in time**, sampling each hidden state x_t conditional on the value x_{t+1} that was just sampled:

$$\text{Sample } x_t = i \text{ with probability } p(x_t = i) = \frac{\alpha(t, i) \cdot \nu(i, x_{t+1})}{\sum_j \alpha(t, j) \cdot \nu(j, x_{t+1})}$$

for $t = n - 1, n - 2, \dots, 1$

In the end, this **“forward summing, backwards sampling”** algorithm gives an entire sampled path $\mathbf{x}$ through the hidden states

Once we have a **sampled path through the hidden states** $\mathbf{x} = (x_1, x_2, \dots, x_n)$, sampling our model parameters is easy

The **conditional posterior** of our **emission model parameters**

$$\theta \sim p(\theta | \mathbf{x}, \nu, \pi, \mathbf{y})$$

depends on our assumed emission model. e.g. for our normal mixture model with different means θ_k for each state k , then

$$\mu_k | \mathbf{x}, \mathbf{y} \sim \text{Normal} \left(\frac{\sum_i \mathbb{I}(x_i = k) y_i}{\sum_i \mathbb{I}(x_i = k)}, \frac{\sigma_1^2}{\sum_i \mathbb{I}(x_i = k)} \right)$$

just like we saw for our mixture model earlier this semester

Once we have a **sampled path through the hidden states** $\mathbf{x} = (x_1, x_2, \dots, x_n)$, sampling our model parameters is easy

Our **transition parameters** ν are a 2×2 matrix where each row has a conditional posterior of

$$(\nu_{j1}, \nu_{j2}) \sim \text{Beta}(N_{j1} + 1, N_{j2} + 1)$$

where $N_{jk} = \sum_t \mathbb{I}(x_t = k \mid x_{t-1} = j)$

If there are more than 2 hidden states, this conditional posterior is a **Dirichlet** distribution (an extension of the Beta distribution)

We also need to sample the **initial state parameters** $\pi = (\pi_1, \pi_2)$ for our 2 hidden states,

$$(\pi_1, \pi_2) \sim \text{Beta}(N_1 + 1, N_2 + 1)$$

where $N_k = \sum_t I(x_t = k)$. This is also a Dirichlet instead of Beta if there are more than 2 states

Work in R: Gibbs Sampling for HMM Examples

There is also an **EM algorithm** for estimating HMMs called the **Baum-Welch algorithm**

Next Class:

Non-Parametric Bayesian Modeling

(Chapter 23 in Gelman Book)